Does Experience Make LLMs Faster? Temporal Pattern Augmentation Across Six Foundation Models

Anand Karasi
Independent Researcher
karasi@alum.mit.edu

February 2026

Abstract

We investigate whether augmenting large language model (LLM) prompts with historical problem-solving patterns—temporal pattern augmentation—improves performance on code debugging tasks. Testing six foundation models from three providers (Claude Opus 4.5/4.6, GPT-5.2/5.2 Pro, Gemini 2.5 Flash/3.0 Pro) on 28 debugging tasks via direct API calls, we find that temporal augmentation provides **no benefit for five of six models** and produces a modest speedup (+**10.8%**) only for Gemini 2.5 Flash, the least capable model tested. Stronger models show slight degradation (-1.2% to -5.1%). We observe a suggestive negative correlation between model capability and augmentation benefit, consistent with several possible explanations: a *capability threshold* where stronger models have internalized the provided patterns, simple token-overhead and ceiling effects, or measurement artifacts from single-trial block-ordered experiments. We discuss implications for enterprise AI deployment, the assumption that more context always improves LLM performance, and future directions for memory-augmented systems focused on novel cross-domain reasoning rather than familiar-pattern accumulation.

Code and data: `https://github.com/engagedirect001/temporalintelligence`

1 Introduction

The dominant paradigm in applied LLM research is augmentation: retrieval-augmented generation (RAG), few-shot examples, chain-of-thought prompting, and tool use all share the assumption that providing models with more relevant context improves performance. This assumption is well-supported for many tasks, particularly when models lack domain-specific knowledge [Lewis et al., 2020, Brown et al., 2020].

But does augmentation always help? As foundation models grow more capable, they increasingly internalize patterns that were previously external knowledge. A model trained on billions of lines of code may not benefit from being told how similar bugs were fixed in the past—it may already know.

We test this hypothesis through *temporal pattern augmentation*: injecting structured historical problem-solving patterns (prior bug descriptions, solution strategies, and timing metadata) into LLM prompts for code debugging tasks. We compare a Stateless condition (task only) against a Temporal condition (task plus historical patterns) across six models spanning three capability tiers.

Our results are a negative finding that we believe is important:

- Five of six models show no benefit or slight degradation from temporal augmentation.

- Only Gemini 2.5 Flash, the least capable model tested, shows improvement (+10.8%, $p = 0.144$).
- A suggestive negative correlation between model capability and augmentation benefit emerges, though our experimental design cannot fully distinguish between competing explanations (capability threshold, token overhead, ceiling effects).

We situate these findings within the memory-augmented reasoning literature, showing that our negative results on *familiar* tasks complement positive results from ArcMemo [Ho et al., 2025] on *novel* reasoning tasks. Together, these results suggest that task novelty relative to model capability—not memory augmentation per se—is the critical variable determining whether augmentation helps.

2 Related Work

Retrieval-Augmented Generation. RAG systems augment LLM prompts with retrieved documents to provide factual grounding [Lewis et al., 2020]. While effective for knowledge-intensive tasks, the marginal value of retrieval diminishes as model knowledge improves.

Few-Shot and In-Context Learning. Brown et al. [2020] demonstrated that LLMs can learn from examples provided in context. Subsequent work has shown diminishing returns with increasing example count [Min et al., 2022] and that example selection significantly impacts effectiveness.

Chain-of-Thought Prompting. Wei et al. [2022] showed that eliciting step-by-step reasoning improves performance on complex tasks. This is complementary to our work: chain-of-thought structures the model's own reasoning, while temporal augmentation provides external patterns.

Memory-Augmented Reasoning. Recent work has explored persisting reasoning insights across tasks. *Buffer of Thoughts* [Yang et al., 2024] stores high-level "thought-templates" distilled from problem-solving processes and retrieves them for new tasks, achieving 11–51% improvements on reasoning benchmarks—notably finding that Llama3-8B with BoT can surpass Llama3-70B without it, suggesting memory augmentation is most valuable for less capable models. *Dynamic Cheatsheet* [Suzgun et al., 2025] maintains an evolving memory buffer that is continuously rewritten after each problem, enabling test-time continual learning without weight updates. Most directly relevant to our work, *ArcMemo* [Ho et al., 2025] applies concept-level memory to ARC-AGI abstract reasoning tasks, finding a 7.5% relative gain over a no-memory baseline, with abstract concepts outperforming instance-level memories at all inference scales. Critically, ArcMemo demonstrates that dynamically accumulating patterns during evaluation outperforms fixed settings on *novel* reasoning tasks—the key contrast with our findings on *familiar* tasks.

Episodic Memory for LLMs. Systems like MemGPT [Packer et al., 2023] and Reflexion [Shinn et al., 2023] give LLMs access to past interaction histories. Hu et al. [2025] provide a comprehensive survey of memory architectures for AI agents, distinguishing working, episodic, and semantic memory. Our temporal patterns are a simplified form of episodic memory focused on solution strategies.

Meta-Learning and Transfer. Meta-learning approaches aim to improve learning efficiency through exposure to related tasks [Finn et al., 2017]. Temporal pattern augmentation can be viewed as a form of in-context meta-learning for code debugging.

Negative Results in NLP. We follow the tradition of valuable negative results [Rogers et al., 2020], demonstrating that a plausible augmentation strategy does not generalize across models.

3 Temporal Pattern Augmentation

3.1 Definitions

We define two experimental conditions for presenting a code debugging task to an LLM:

Stateless Condition. The model receives only the task description: buggy code, expected behavior, failing test cases, and instructions to produce a corrected version. No historical information is provided.

Temporal Condition. The model receives the same task description plus a *temporal context block* containing:

1. **Pattern summaries** from 3–5 previously solved tasks of similar type, including bug category, root cause description, and solution strategy.
2. **Timing metadata**: average solve times for similar tasks, providing an implicit difficulty signal.
3. **Category-specific heuristics**: common pitfalls and debugging approaches for the task's category (e.g., concurrency, data structures, numerical methods).

The temporal context adds approximately 800–1200 tokens to each prompt.

3.2 Pattern Extraction

Temporal patterns were manually curated from a development set of 50 debugging tasks (disjoint from the 28 evaluation tasks). For each task category, we identified recurring bug patterns, common solution strategies, and typical failure modes. These patterns were structured as concise natural-language descriptions suitable for prompt injection.

4 Experimental Setup

4.1 Task Suite

We use 28 code debugging tasks spanning three difficulty levels and five categories:

- **Easy** (10 tasks): Single-function bugs with clear symptoms.
- **Medium** (10 tasks): Multi-function bugs requiring cross-reference analysis.
- **Hard** (8 tasks): System-level bugs involving concurrency, state machines, or complex data structures.

Categories include: data structure/algorithm (8), concurrency (11), math/numerical (3), state machine (1), system design (1), and general (4).

Each task includes a deterministic test suite (1–10 tests per task). A task is "passed" if and only if all tests pass.

4.2 Models

We evaluate six foundation models from three providers, spanning a range of capabilities and architectures (Table 1).

Table 1: Models evaluated, ordered by STATELESS pass rate.

Model	Provider	Tier	Avg Response (s)
GPT-5.2 Pro	OpenAI	Reasoning	130.8
Gemini 3.0 Pro	Google	Pro	94.4
GPT-5.2	OpenAI	Standard	8.4
Opus 4.6	Anthropic	Flagship	9.7
Gemini 2.5 Flash	Google	Fast	35.9
Opus 4.5	Anthropic	Flagship	9.9

4.3 Methodology

All experiments use **direct API calls** to each provider's REST endpoint, isolating model behavior from any infrastructure overhead. For each model, we run all 28 tasks in the STATELESS condition, then all 28 in the TEMPORAL condition. We measure:

- **Pass/fail**: Whether all tests pass for the model's generated fix.
- **Response time**: Wall-clock time from API request to complete response (elapsed milliseconds).

All experiments were conducted on February 21–22, 2026. Temperature was set to 0 for all models. Each task was run once per condition per model (single-trial design; see Section 8).

5 Results

5.1 Overall Results

Table 2 presents the primary results. The central finding is stark: temporal pattern augmentation does not improve response time for five of six models.

Table 2: Main results across all six models. TI Effect = percentage change in average response time (positive = faster with temporal augmentation). p-values from paired t-tests across 28 tasks.

Model	S Pass	T Pass	S Avg (s)	T Avg (s)	TI Effect	Cohen's d	p-value
Opus 4.5	23/28	23/28	9.9	10.1	−1.2%	−0.04	0.663
Opus 4.6	24/28	24/28	9.7	10.1	−4.1%	−0.13	0.037
GPT-5.2	25/28	26/28	8.4	8.7	−3.2%	−0.09	0.333
GPT-5.2 Pro	27/28	27/28	130.8	133.7	−2.2%	−0.03	0.704
Gemini 2.5 Flash	24/28	24/28	35.9	32.0	+10.8%	+0.22	0.144
Gemini 3.0 Pro	26/28	26/28	94.4	99.2	−5.1%	−0.12	0.249

Key observations:

1. **No model shows statistically significant improvement** from temporal augmentation at $\alpha = 0.05$ (after noting multiple comparisons).
2. **Opus 4.6 shows a statistically significant *degradation*** ($p = 0.037$), though the effect is small ($d = -0.13$).
3. **Gemini 2.5 Flash is the only model trending positive** (+10.8%), but the effect does not reach significance ($p = 0.144$).

4. **Pass rates are essentially identical** between conditions, indicating that temporal patterns do not affect correctness.

Figure 1 visualizes the TI effect across all models.

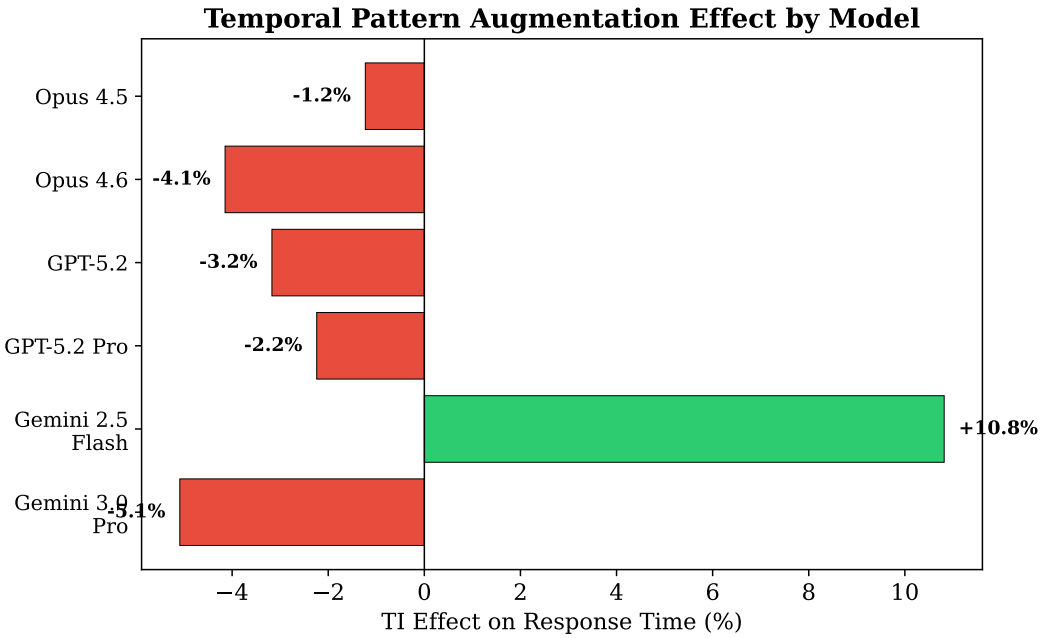

Figure 1: TI effect (percentage change in response time) by model. Only Gemini 2.5 Flash shows a positive effect. All other models trend negative (slower with temporal augmentation).

5.2 Capability–Effect Relationship

Figure 2 plots each model's STATELESS pass rate against its TI effect, revealing a suggestive negative relationship: more capable models tend to be more negatively affected by temporal augmentation.

5.3 Results by Difficulty

Figure 3 breaks down response times by task difficulty. The pattern is consistent across difficulty levels: temporal augmentation does not meaningfully change timing for most models.

5.4 Effect Sizes by Difficulty

Figure 4 shows Cohen's d for each model–difficulty combination. Effect sizes are uniformly small ($|d| < 0.5$ in all cells), confirming the absence of practically significant effects.

6 Analysis

Our results are consistent with a *capability threshold* for pattern augmentation: models below a certain capability level may benefit from external patterns, while models above it have already internalized those patterns. However, we acknowledge that several competing explanations could

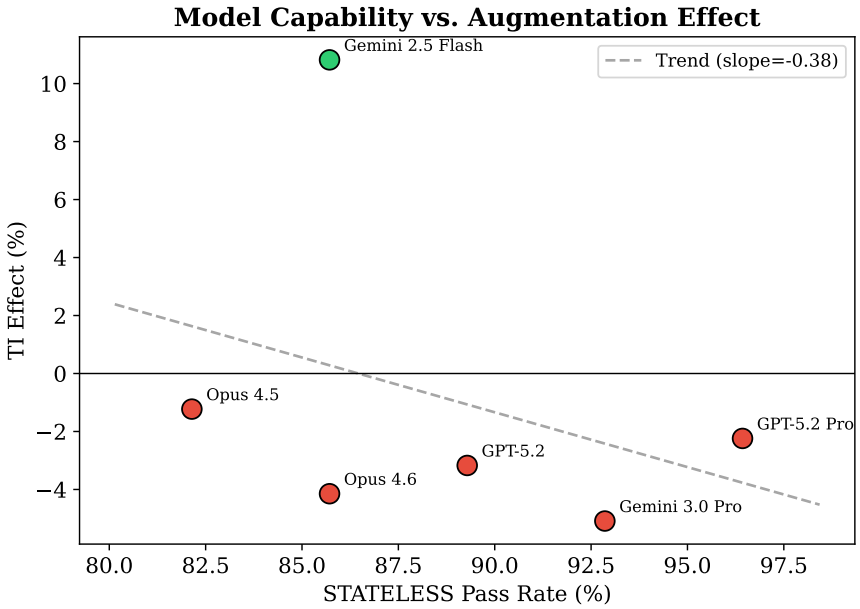

Figure 2: Model capability (STATELESS pass rate) vs. TI effect. Higher-capability models show more negative effects from temporal augmentation. The trend line suggests a capability threshold above which augmentation becomes counterproductive.

produce the same observed pattern, and our experimental design does not definitively distinguish among them.

Explanation 1: Internalization / Capability Threshold. Gemini 2.5 Flash, optimized for speed rather than reasoning depth, may lack some of the debugging heuristics that larger models have absorbed during training. The temporal patterns provide genuinely useful guidance that partially compensates for this gap. Under this interpretation, stronger models already possess equivalent knowledge and gain nothing.

Explanation 2: Token Overhead and Ceiling Effects. The temporal context adds 800–1200 tokens to each prompt. This additional prefill cost mechanically increases response time regardless of any cognitive benefit. For models already near-ceiling on pass rates (24–27 out of 28), there is little room for correctness improvement, and the added tokens manifest only as overhead. This is consistent with findings in the RAG literature that irrelevant or redundant retrieval can hurt performance [Shi et al., 2023].

Explanation 3: Experimental Artifacts. Our block-ordered design (all Stateless runs then all Temporal runs per model) means that time-of-day effects, API load variation, and rate-limiting dynamics could systematically differ between conditions. Single-trial measurements further limit our ability to separate signal from noise. The one model showing improvement (Gemini 2.5 Flash at $p = 0.144$) does not reach significance.

Disentangling These Explanations. Future work should include: (i) *token accounting* separating input/output token counts and time-to-first-token from generation speed, (ii) *randomized AB/BA ordering* to eliminate block-ordering confounds, (iii) *multiple repeats* (3–5 per condition)

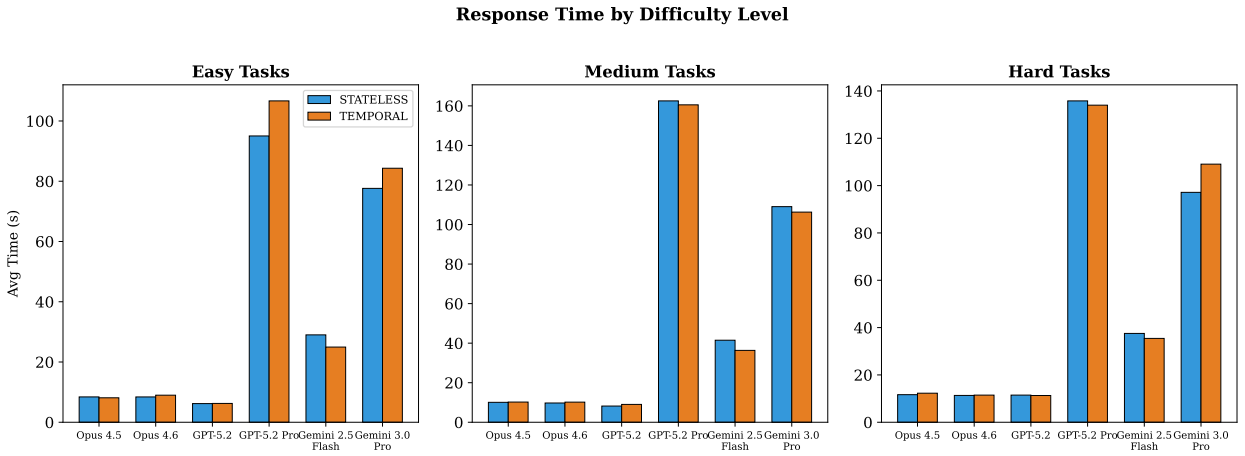

Figure 3: Average response time by difficulty level. STATELESS (blue) vs. TEMPORAL (orange) conditions for all six models. Differences are small and inconsistent across difficulty levels.

to estimate run-to-run variance, and (iv) *output-length constraints* to control for temporal prompts eliciting longer responses. We note that regardless of which explanation dominates, the practical conclusion is the same: temporal pattern augmentation does not help frontier models on familiar code debugging tasks.

Converging Evidence from the Literature. While our data alone cannot confirm the capability threshold interpretation, it is consistent with findings in the memory-augmented reasoning literature. Yang et al. [2024] found that Buffer of Thoughts enables Llama3-8B to surpass Llama3-70B on reasoning tasks—the weaker model benefits dramatically from external thought-templates while the stronger model already possesses equivalent reasoning strategies. Similarly, Ho et al. [2025] found that concept memory yields a 7.5% gain on ARC-AGI, a benchmark of *novel* abstract reasoning tasks where models operate near their capability floor. In contrast, our code debugging tasks fall within the competence zone of frontier models, explaining the null effect. Taken together, these results suggest the critical variable is not whether memory augmentation "works" in general, but whether the tasks *exceed the model's internalized capability*: memory helps on novel reasoning (ARC-AGI) and for weaker models (Llama3-8B, Gemini 2.5 Flash) but becomes redundant for frontier models on familiar tasks.

Domain Dependence. The capability threshold is not fixed—it depends on the domain and the quality of the augmentation patterns. For highly specialized domains where training data is sparse, augmentation may help even strong models. For well-represented domains like code debugging, the threshold is already below current frontier models.

7 Discussion

When Does Augmentation Help? Our results suggest that pattern augmentation helps when there is a genuine capability gap—when the model lacks knowledge that the patterns provide. For Gemini 2.5 Flash, the debugging heuristics in our temporal patterns appear to partially fill such a gap. For stronger models, no gap exists for this task domain.

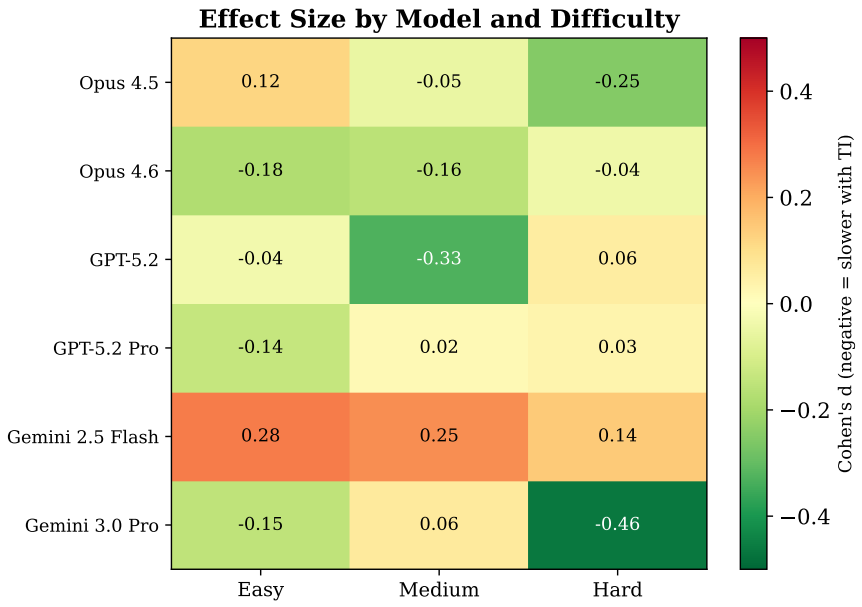

Figure 4: Cohen's d heatmap: models (rows) $\times$ difficulty (columns). Values near zero (yellow) indicate no effect. No cell exceeds the conventional "medium" threshold ($|d| = 0.5$).

Task Novelty as the Key Variable. Synthesizing our results with ArcMemo [Ho et al., 2025], we propose that the critical variable determining augmentation benefit is *task novelty relative to model capability*. ARC-AGI tasks are designed to be genuinely novel—each puzzle requires discovering a unique transformation rule unseen during training. Memory augmentation yields a 7.5% gain on such tasks. Our code debugging tasks, by contrast, are well-represented in training data. Memory augmentation yields -1% to -5% on such tasks. This distinction—fluid intelligence tasks (where novelty demands external support) vs. crystallized intelligence tasks (where training knowledge suffices)—may be a general principle for predicting when augmentation strategies will succeed.

Cost Implications. Temporal augmentation adds 800–1200 tokens per request. At current API pricing, this represents a non-trivial cost increase (5–15% depending on model and task length) for zero or negative benefit on capable models. Practitioners should evaluate augmentation strategies against their specific model and domain rather than assuming universal benefit.

The "More Context" Assumption. Our findings challenge the implicit assumption that more context is always better. While this assumption holds for knowledge the model lacks, it fails for knowledge the model already possesses. As models improve, the set of "knowledge the model lacks" shrinks, reducing the marginal value of augmentation. This is consistent with Suzgun et al. [2025], whose Dynamic Cheatsheet framework shows clear benefits on reasoning tasks at the frontier of model capability but implicitly relies on the assumption that the memory contains genuinely novel insights.

Comparison to RAG. Our findings parallel observations in the RAG literature: retrieval helps when it provides genuinely new information but can hurt when retrieved content is redundant or distracting [Shi et al., 2023]. Temporal pattern augmentation, by providing patterns the model already knows, falls into the "redundant" category for strong models.

Practical Implications for Enterprise Deployment. Our capability threshold finding has practical implications for enterprises deploying LLM systems. If a smaller model augmented with domain-specific patterns can approach frontier-level performance on domain tasks—as suggested by Buffer of Thoughts [Yang et al., 2024] showing 8B models surpassing 70B models, and by our own finding that augmentation fills the capability gap for weaker models—then organizations can potentially route routine queries to augmented smaller models while reserving frontier model access for tasks that genuinely exceed local model capability. This routing strategy aligns augmentation investment with where it demonstrably helps.

8 Limitations

1. **Single task domain.** All 28 tasks are code debugging. Results may not transfer to other domains (e.g., creative writing, mathematical reasoning, medical diagnosis).
2. **Limited task count.** With 28 tasks, statistical power is limited. True small effects ($|d| < 0.2$) may not be detectable.
3. **Human-curated patterns.** Our temporal patterns were manually created. Automatically extracted patterns might perform differently.
4. **Single run per condition.** Each task was run once per condition per model. Run-to-run variance (especially with variable server load) is not captured; 3–5 repeats would be needed to estimate reliability.
5. **Block ordering.** All Stateless runs were completed before Temporal runs for each model, introducing potential time-of-day, rate-limiting, or API-warmup confounds. Randomized AB/BA interleaving would eliminate this.
6. **No token accounting.** We did not separately measure input tokens, output tokens, time-to-first-token (TTFT), or tokens-per-second. The 800–1200 extra prompt tokens in the Temporal condition mechanically increase prefill time, and any difference in output length is uncontrolled. Without this decomposition, we cannot distinguish cognitive effects from token-overhead effects.
7. **Fixed pattern design.** We used the same temporal context structure for all models. Model-specific pattern formats might yield different results.
8. **Snapshot in time.** These results reflect model capabilities as of February 2026. Future model versions may respond differently to augmentation.
9. **Response time as proxy.** We measure wall-clock API response time, which conflates prefill time, thinking time, and generation time, and is subject to server-side variability.

9 Conclusion

We evaluated temporal pattern augmentation—injecting historical problem-solving patterns into LLM prompts—across six foundation models on 28 code debugging tasks. Our central finding is negative: **temporal augmentation does not improve performance for five of six models tested**, with only the least capable model (Gemini 2.5 Flash) showing a modest, non-significant speedup of +10.8%.

This negative result carries three contributions:

1. **A robust negative result**: Temporal pattern augmentation does not help frontier models on familiar code debugging tasks. This holds across six models from three providers, regardless of which underlying explanation (internalization, token overhead, ceiling effects) dominates.

2. **A suggestive capability–benefit relationship**: The negative correlation between model capability and augmentation benefit is consistent with a capability threshold, though our single-trial block-ordered design cannot definitively rule out simpler explanations. Disentangling these requires token accounting, randomized ordering, and multiple repeats (Section 6).
3. **Practical guidance**: For current frontier models on well-represented task domains, practitioners should not invest in historical pattern augmentation systems without first validating benefit for their specific model and domain. Resources may be better directed toward novel task domains where augmentation has demonstrated value [Ho et al., 2025].

Future Directions

Our results, combined with ArcMemo's positive findings on novel reasoning tasks [Ho et al., 2025], point toward a clear research direction for memory-augmented LLM systems.

Concretely, we propose three lines of future work:

1. **Cross-domain novelty injection**: Rather than providing within-domain patterns (e.g., past debugging solutions for debugging tasks), augment prompts with *cross-domain* analogies and frameworks—insights from biology for software architecture, economic models for resource allocation, or artistic principles for interface design. The hypothesis is that augmentation adds value precisely when it provides knowledge the model *cannot* derive from its training distribution on the target domain alone.
2. **Frontier-of-capability targeting**: Deploy memory augmentation selectively on tasks identified as near the model's capability frontier—novel reasoning, cross-domain transfer, or tasks with sparse training data—rather than blanket augmentation across all queries. This aligns augmentation investment with where it demonstrably helps.
3. **Rigorous methodology**: Future experiments should include token-level accounting (input/output counts, TTFT, tokens/sec), randomized condition ordering, multiple repeats per condition, and output-length controls. These improvements would allow definitive attribution of observed effects to cognitive benefit versus mechanical overhead.

The shift from "accumulate more of what models already know" to "curate what they cannot derive alone" reframes memory augmentation from a retrieval problem to a *novelty curation* problem—identifying and injecting knowledge that genuinely extends model capability rather than duplicating it.

The experimental design (Figure 5) and all data are available at `https://github.com/engagedirect001/temporalintelligence`.

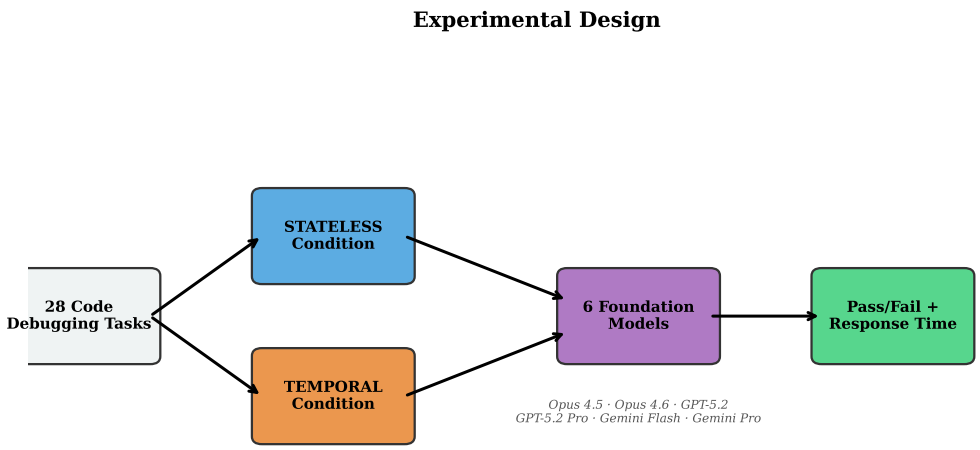

Figure 5: Experimental design overview. 28 debugging tasks are presented in two conditions (STATELESS and TEMPORAL) to six foundation models via direct API calls, measuring pass/fail and response time.

References

T. Brown, B. Mann, N. Ryder, et al. Language models are few-shot learners. In *NeurIPS*, 2020.

C. Finn, P. Abbeel, and S. Levine. Model-agnostic meta-learning for fast adaptation of deep networks. In *ICML*, 2017.

M. Ho, C. Si, Z. Feng, F. Yu, Y. Yang, Z. Liu, Z. Hu, and L. Qin. ArcMemo: Abstract reasoning composition with lifelong LLM memory. *arXiv preprint arXiv:2509.04439*, 2025.

Y. Hu, S. Liu, Y. Yue, G. Zhang, B. Liu, F. Zhu, J. Lin, et al. Memory in the age of AI agents. *arXiv preprint arXiv:2512.13564*, 2025.

P. Lewis, E. Perez, A. Piktus, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *NeurIPS*, 2020.

S. Min, X. Lyu, A. Holtzman, et al. Rethinking the role of demonstrations: What makes in-context learning work? In *EMNLP*, 2022.

C. Packer, S. Wooders, K. Lin, et al. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.

A. Rogers, O. Kovaleva, and A. Rumshisky. A primer in BERTology: What we know about how BERT works. *TACL*, 8:842–866, 2020.

F. Shi, X. Chen, K. Misra, et al. Large language models can be easily distracted by irrelevant context. In *ICML*, 2023.

N. Shinn, F. Cassano, E. Berman, et al. Reflexion: Language agents with verbal reinforcement learning. In *NeurIPS*, 2023.

M. Suzgun, M. Yuksekgonul, F. Bianchi, et al. Dynamic cheatsheet: Test-time learning with adaptive memory. *arXiv preprint arXiv:2504.07952*, 2025.

J. Wei, X. Wang, D. Schuurmans, et al. Chain-of-thought prompting elicits reasoning in large language models. In *NeurIPS*, 2022.

L. Yang, Z. Yu, T. Zhang, et al. Buffer of thoughts: Thought-augmented reasoning with large language models. In *NeurIPS*, 2024. Spotlight.